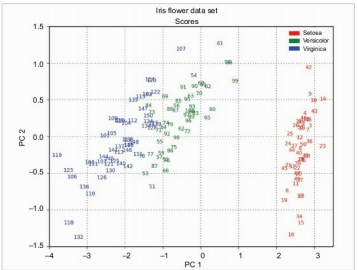

Figure 13: Scores MDP

```
pt.add_column("PCT",p)
pt.add_column("SUM",r)
pt.align="r"
print(pt)
```

The result is a well-formatted table:

```
+----+------+--------+-------+
| N  | VAR  |   PCT  |  SUM  |
+----+------+--------+-------+
| 1  | 4.23 | 92.4619| 92.46 |
| 2  | 0.24 | 5.3066 | 97.77 |
| 3  | 0.08 | 1.7103 | 99.48 |
| 4  | 0.02 | 0.5212 | 100.00|
+----+------+--------+-------+
```

The scree plot is plotted with a simple *bar* plot type (Figure 5), the scores (Figure 6) and the loadings (Figure 7) with *plot*. For the scores, the colours are chosen according to the different iris species, because in this example, the data are already categorised.

A bit more complex is the scores plot with clipart, as shown in Figure 8 as an example. The original clipart is taken from *http://www.worldartsme.com/images/iris-flower-clipart-1.jpg*, and then processed via ImageMagick. Each clipart is read with *imread*, zoomed with *OffsetImage* and then placed on the plot at the scores coordinates with *AnnotationBbox*, according to the following code:

```
import matplotlib.image as imread
from matplotlib.offsetbox import AnnotationBbox,OffsetImage
i1=imread("iris1.png")
i2=imread("iris2.png")
i3=imread("iris3.png")
o=range(1,w.nrows+1)
ax=subplot(111)
for i,j,o in zip(s1,s2,o):
    if o<51: ib=OffsetImage(i1,zoom=0.75)
    elif o>50 and o<101: ib=OffsetImage(i2,zoom=0.75)
    elif o>100: ib=OffsetImage(i3,zoom=0.75)
    ab=AnnotationBbox(ib,[i,j],xybox=None,xycoords="data",fram
eon=False,boxcoords=None)
    ax.add_artist(ab)
```

The two plots about scores and loadings can be overlapped to obtain a particular plot called the biplot. The example presented here is based on a scaling of the scores as in the following code:

```
xS=(1/(max(s1)-min(s1)))*1.15
yS=(1/(max(s2)-min(s2)))*1.15
```

Then the loadings are plotted with *arrow* over the scores, and the result is shown in Figure 9. This solution is based on the one proposed at *http://sukhbinder.wordpress.com/2015/08/05/biplot-with-python;* it probably is not the best way, but it works.

The 3D plots (Figures 10 and 11) do not present any particular problems, and can be done according to the following code:

```
from mpl_toolkits.mplot3d import Axes3D
ax=Axes3D(figure(0),azim=-70,elev=20)
ax.scatter(s1,s2,s3,marker="")
for i,j,h,o in zip(s1,s2,s3,o):
    if o<51: k="r"
    elif o>50 and o<101: k="g"
    elif o>100: k="b"
    ax.text(i,j,h,"%.0f"%o,color=k,ha="center",va="center",fo
ntsize=8)
```

Using the singular value decomposition (SVD) is very easy—just call *pcasvd* on the scaled data. The result is shown in Figure 12.

```
from statsmodels.sandbox.tools.tools_pca import pcasvd
xreduced,scores,evals,evecs=pcasvd(dataS)
```

The modular toolkit for the data processing (MDP) package (see References 4 and 5) is not included in WinPython; so it's necessary to download the source *MDP-3.5.tar.gz* from *https://pypi.python.org/pypi/MDP*. Then open the WinPython control panel and go to the *install/upgrade packages* tab. Drag the source file and drop it there. Click on 'Install packages'. Last, test the installation with the following command:

```
import mdp
mdp.test()
```

This is a bit time consuming; another test is the following command:

```
import bimdp
bimdp.test()
```